Tutoriumsblatt 7

RAG: Privates Wissen nutzbar machen

Bearbeitungszeit etwa 90 Minuten · 40 Punkte · Lösungen im Anhang ab Seite 6

Nach diesem Blatt kannst du

- die Kosten von „alles in den Prompt“ und RAG vergleichen,
- die Zahl der Chunks mit Überlappung herleiten und die Abdeckung prüfen,
- Kosinus-Ranglisten berechnen und Bi- und Cross-Encoder vergleichen,
- BM25 von Hand ausrechnen und Sättigung und Längennormierung erklären,
- Ranglisten mit RRF verbinden, Recall@k und MRR berechnen, Fehler zuordnen.

So arbeitest du mit dem Blatt

Bearbeite die Aufgaben der Reihe nach, sie werden schwieriger. Taschenrechner mit ln genügt. Rechne Zwischenergebnisse mit mindestens drei Nachkommastellen (RRF-Punkte in Aufgabe 5 mit fünf) und runde erst die Endergebnisse. Alle Zahlen sind ausgedachte Beispiele, sofern nicht anders angegeben. Bezug zum Notebook: Aufgabe 2 gehört zu TODO 1 (`chunk_fest`), Aufgabe 3 zu TODO 4 und 7 (`suche`, `rerank`), Aufgabe 4 und 5(a) zu Bonus B1, Aufgabe 5 zu TODO 5 und 8. Dort programmierst du, hier rechnest du von Hand.

Aufgabe 1 Alles in den Prompt oder RAG?

5 Punkte · etwa 10 Min.

Bezug: Modul 7, Lerneinheit 1 (Warum nicht alles in den Kontext?)

Die Instandhaltung der Nordwerk GmbH will Fragen zur gesamten technischen Dokumentation beantworten lassen: Wartungshandbuch und Herstellerunterlagen aller acht Maschinen, zusammen 400 Seiten. Rechne wie in der Lerneinheit mit etwa 750 Tokens pro Seite und dem kommerziellen Referenzpreis von 1 US-Dollar pro Million Input-Tokens. Erwartet werden 1200 Fragen pro Monat.

(a) *Alles in den Prompt:* Berechne die Input-Tokens pro Frage sowie die Input-Kosten pro Frage und pro Monat. [1 P]

(b) *RAG:* Pro Frage gehen $k = 4$ Chunks mit je etwa 200 Tokens in den Prompt, dazu 600 Tokens für System-Prompt, Frage und XML-Tags. Berechne die Input-Kosten pro Frage und pro Monat und den Faktor, um den RAG weniger Input-Tokens braucht. Begründe kurz, warum du die Output-Tokens für diesen Vergleich weglassen darfst. [2 P]

(c) Die Abteilungsleiterin sagt: „Das Geld haben wir. Wir legen einfach alles in den Prompt.“ Nenne zwei Gründe, die trotzdem für RAG sprechen, mit je einem Satz Begründung. [2 P]

Aufgabe 2 Chunking mit Überlappung

8 Punkte · etwa 17 Min.

Bezug: Modul 7, Lerneinheit 2 (Chunking: Wie schneidet man ein Handbuch?)

Ein Text mit L Zeichen wird in Chunks der Größe g mit Überlappung o geschnitten, die Schrittweite ist $s = g - o$. Die Zeichen sind ab 0 nummeriert. Chunk j ($j = 0, 1, 2, \dots$) beginnt bei Position js und enthält die Positionen js bis $\min(js + g,\ L) - 1$. Wie in `chunk_fest` hängt die Schleife Chunk j an und hört beim ersten j mit $js + g \geq L$ auf. Dieser Chunk ist also der letzte.

(a) Leite aus der Stoppbedingung her, dass für $L > g$ genau

$$n = \left\lceil \frac{L - g}{s} \right\rceil + 1$$

Chunks entstehen. Erkläre, woher die $+1$ kommt und warum $o < g$ gelten muss. [2 P]

(b) Eine Herstellerunterlage der Rundschleifmaschine M-04 hat $L = 5000$ Zeichen. Geschnitten wird mit $g = 600$ und $o = 100$. Berechne n, alle Starts und die Länge des letzten Chunks. [2 P]

(c) *Abdeckung:* Wie viele Zeichen werden insgesamt eingebettet, also wie groß ist die Summe aller Chunk-Längen? Um welchen Faktor ist das mehr als L? Wie viele Zeichen des Texts stehen in genau zwei Chunks? [2 P]

(d) Zeige: Jede zusammenhängende Textstelle mit höchstens o Zeichen liegt vollständig in mindestens einem Chunk. Prüfe dann mit den Zahlen aus (b): Ein Satz mit 120 Zeichen beginnt bei Position 490. Liegt er vollständig in einem Chunk? Welche Überlappung garantiert nach deinem Beweis, dass kein Satz mit bis zu 120 Zeichen zerschnitten wird? [2 P]

> **Tipp**
> Zu (d): Beginnt die Stelle bei a, nimm Chunk $j = \min\left(\lfloor a/s \rfloor,\ n - 1\right)$. Behandle den letzten Chunk getrennt.

Aufgabe 3 Mit Kosinus suchen, dann genauer sortieren

8 Punkte · etwa 16 Min.

Bezug: Modul 7, Lerneinheiten 3 (Suchen mit Embeddings und Kosinus) und 4 (Bi-Encoder und Cross-Encoder)

Gegeben sind ein normierter Frage-Vektor q und vier normierte Chunk-Vektoren als Zeilen der Matrix E. Zeile i ist e_i, der Einfachheit halber mit drei statt 384 Dimensionen:

$$E = \begin{pmatrix} 0{,}8 & 0{,}6 & 0 \\ 0 & 0{,}6 & 0{,}8 \\ 0{,}48 & 0{,}6 & 0{,}64 \\ 0{,}6 & 0{,}8 & 0 \end{pmatrix}, \qquad q = \begin{pmatrix} 0{,}6 \\ 0 \\ 0{,}8 \end{pmatrix}.$$

(a) Prüfe, dass e_3 die Länge 1 hat. Berechne $s = E\,q$ und gib die Rangfolge der Chunks an. [2 P]

(b) Bei einem fünften Chunk wurde die Normierung vergessen: $e_5 = (1{,}2;\ 1{,}6;\ 0)$. Berechne $q \cdot e_5$ und $\cos(q, e_5)$. Auf welchem Platz landet Chunk 5, wenn die Suche wie in TODO 4 nur das Skalarprodukt verwendet? Auf welchen Platz gehört er? [2 P]

(c) *Rechenaufwand:* Eine Wissensbasis hat 120 000 Chunks. Ein Cross-Encoder braucht 15 ms pro Paar aus Frage und Chunk. Die erste Stufe (Frage einbetten, Vektorsuche) dauert zusammen 40 ms. (i) Wie lange dauert eine Frage, wenn der Cross-Encoder alle Chunks bewertet? (ii) Wie lange dauert sie zweistufig mit 25 Kandidaten? (iii) Wie viele Kandidaten darf die zweite Stufe höchstens bekommen, wenn das ganze Retrieval höchstens 1 s dauern soll? [3 P]

(d) Begründe: Wer 25 Kandidaten neu sortiert, kann nach dem Reranking keinen höheren Recall@5 erreichen als den Recall@25 der ersten Stufe. [1 P]

Aufgabe 4 BM25 von Hand

9 Punkte · etwa 20 Min.

Bezug: Modul 7, Lerneinheit 5 (BM25 und hybride Suche)

Ein ausgedachter Mini-Index hat $N = 10$ Chunks mit der mittleren Länge avgdl = 80 Wörter. Die Frage „E21 Kühlflüssigkeit prüfen“ wird wie in Bonus B1 in kleingeschriebene Wörter zerlegt: *e21*, *kühlflüssigkeit* und *prüfen*. Die Tabelle zeigt für drei Chunks die Länge $|D|$ in Wörtern und die Termfrequenzen, in der letzten Zeile die Zahl n_t der Chunks, die das Wort enthalten.

Chunk	$\lvert D\rvert$	e21	kühlflüssigkeit	prüfen
C1: 5.1 Fehlercodes der Schweißstromquelle	120	1	1	5
C2: 3.5 M-05, Wartungsintervalle	80	0	2	3
C3: Checkliste Schichtbeginn	160	0	0	8
n_t (Zahl der Chunks mit dem Wort t)		1	2	4

Zur Erinnerung: TF-IDF nutzt $\mathrm{idf}(t) = \ln(N/n_t)$ und $\mathrm{Score}(D) = \sum_{t\in Q} \mathrm{tf}(t,D)\cdot\mathrm{idf}(t)$. BM25 rechnet mit $k_1 = 1{,}5$ und $b = 0{,}75$:

$$\text{BM25:}\quad \mathrm{BM25}(D,Q) = \sum_{t\in Q} \mathrm{idf}_{\mathrm{BM25}}(t)\cdot f(\mathrm{tf}(t,D)), \qquad \mathrm{idf}_{\mathrm{BM25}}(t) = \ln\frac{N-n_t+0{,}5}{n_t+0{,}5},$$

$$f(\mathrm{tf}) = \frac{\mathrm{tf}\cdot(k_1+1)}{\mathrm{tf} + k_1\left(1-b+b\,\frac{|D|}{\mathrm{avgdl}}\right)}.$$

(a) Berechne für alle drei Wörter beide idf-Werte. Welches Wort zählt am meisten? [2 P]

(b) Berechne die TF-IDF-Scores der drei Chunks und ihre Rangfolge. [1 P]

(c) Berechne die BM25-Scores und die Rangfolge. Zur Kontrolle: Für C1 ist $1-b+b\,|D|/\mathrm{avgdl} = 1{,}375$ und $\mathrm{BM25}(\mathrm{C1}) \approx 3{,}16$. [3 P]

(d) C2 und C3 tauschen beim Übergang zu BM25 die Plätze. Liegt das an der anderen idf-Formel oder am Faktor f? Rechne zur Trennung beide Chunks mit den einfachen Gewichten $\ln(N/n_t)$, aber mit den Faktoren f aus (c). [1 P]

(e) Zeige für den Faktor f: (i) Für $|D| = \mathrm{avgdl}$ ist $f(1) = 1$. (ii) Für $\mathrm{tf} \to \infty$ strebt f gegen $k_1 + 1$. (iii) Wie lang muss ein Chunk im Verhältnis zu avgdl sein, damit ein einzelnes Vorkommen nur noch $f(1) = 0{,}5$ zählt? [2 P]

Aufgabe 5 Hybride Suche messen und Fehler finden

10 Punkte · etwa 24 Min.

Bezug: Modul 7, Lerneinheiten 5 bis 7 (Reciprocal Rank Fusion, Recall@k und MRR, Fehlersuche)

Ein Team prüft am Wartungshandbuch, ob die hybride Suche besser ist als die reine Vektorsuche. In den Prompt gehen jeweils $k = 4$ Chunks. Für die Frage F2 „Was bedeutet E45?“ liefern beide Suchen diese Top 5:

Rang	1	2	3	4	5
Vektorsuche	S	K	W	L	F
BM25	P	F	S	Y	W

Relevant ist nur F, die Fehlercode-Tabelle der Schweißstromquelle (Abschnitt 5.1). Die anderen Chunks sind S, W und L (zu M-05: Typische Störungen, Wartungsintervalle, Kurzbeschreibung), K (5.2 Kontakte), P (1.4 Dokumentation von Störungen und Wartungen) und Y (4 Ersatzteillager und Bestellung).

(a) Berechne mit Reciprocal Rank Fusion (Konstante 60) die Punkte aller sieben Chunks auf fünf Nachkommastellen und die fusionierte Rangfolge. Vereinfachend gehen nur diese Top 5 in die Fusion ein, ein fehlender Eintrag bringt 0 Punkte. Landet F unter den $k = 4$ Chunks im Prompt? Und mit der Vektorsuche allein? [3 P]

(b) Das Testset hat sechs Fragen mit je genau einem relevanten Chunk. Die Tabelle nennt seinen Rang, „fehlt“ heißt: nicht unter den Top 10. Berechne Recall@1, Recall@4 und MRR für beide Verfahren. [3 P]

Frage	Thema	Vektorsuche	Hybrid (RRF)
F1	Spindellager der Drehmaschine nachschmieren	1	1
F2	Was bedeutet E45?	5	2
F3	Konzentration des Kühlschmierstoffs	2	3
F4	Ersatzteil HTD-3310 und Lagerort	fehlt	7
F5	Rufbereitschaft nachts	3	2
F6	Klangprobe der Schleifscheibe	1	1

(c) Ins Protokoll schreibt jemand: „Hybrid ist besser, Recall@4 steigt von 67 % auf 83 %.“ Wie viele Prozentpunkte macht eine einzelne Frage aus? Ist der Befund belastbar? Was spricht inhaltlich trotzdem für die hybride Suche, und was tust du als Nächstes? [2 P]

(d) Ordne jeden Fall einer Stufe der Pipeline zu (Chunking, Retrieval, Auswahl, Generierung). Begründe und nenne eine passende Abhilfe.

Fall 1: Frage F4 mit hybrider Suche und $k = 4$. Die Antwort lautet „Dazu steht nichts im Handbuch.“

Fall 2: Frage F1. Quelle Q1 ist der Abschnitt „M-03 CNC-Drehmaschine › Wartungsintervalle“ mit den Zeilen „Spindellager mit Wälzlagerfett NW-LX2 nachschmieren, Menge 8 Gramm je Lager | alle 500 Betriebsstunden“ und „Spannfutter zerlegen, reinigen, Backen fetten | alle 250 Betriebsstunden“. Antwort: „Alle 250 Betriebsstunden [Q1].“ [2 P]

Aufgabe 6 (⋆) Wozu die 60 in Reciprocal Rank Fusion?

Zusatz · etwa 10 Min.

Vertiefung über das Modul hinaus, mit einer Taylor-Näherung aus der Analysis. Ersetze die 60 durch eine allgemeine Konstante $c > 0$: $\mathrm{RRF}_c(d) = \sum_r 1/(c + \mathrm{rang}_r(d))$. Chunk P steht in Liste 1 auf Rang 1 und in Liste 2 auf Rang 8. Chunk Q steht in beiden Listen auf Rang 3.

(a) Zeige: Q liegt genau dann vor P, wenn $c > 11/3$ gilt. Prüfe das Ergebnis für $c = 0$ und $c = 60$.

(b) Begründe mit der Näherung $\frac{1}{c+r} \approx \frac{1}{c} - \frac{r}{c^2}$ für $c \gg r$: Bei großer Konstante zählt zuerst, in wie vielen Listen ein Chunk vorkommt, und danach die Summe seiner Ränge. Prüfe diese Faustregel an der fusionierten Rangfolge aus Aufgabe 5(a).

Anhang: Lösungen

Erst selbst rechnen, dann vergleichen. Die Lösungen zeigen den Rechenweg und typische Fehler.

Lösung zu Aufgabe 1 Alles in den Prompt oder RAG?

(a) Input pro Frage: $400 \cdot 750 = 300\,000$ Tokens. Mit der Kostenformel aus Modul 2:

$$K = \frac{n_{\text{in}} \cdot p_{\text{in}}}{10^6} = \frac{300\,000 \cdot 1 \text{ US-Dollar}}{10^6} = \boxed{0{,}30 \text{ US-Dollar pro Frage}}.$$

Pro Monat: $1200 \cdot 0{,}30 = \boxed{360 \text{ US-Dollar}}$.

(b) Input pro Frage: $4 \cdot 200 + 600 = 1400$ Tokens, also $1400/10^6 = \boxed{0{,}0014 \text{ US-Dollar}}$ pro Frage und $1200 \cdot 0{,}0014 = \boxed{1{,}68 \text{ US-Dollar}}$ pro Monat. Faktor: $300\,000/1400 \approx \boxed{214}$.

Die Output-Tokens darfst du weglassen, weil die Antwort in beiden Varianten etwa gleich lang ist (höchstens wenige Sätze). Ihre Kosten fallen also in beiden Fällen gleich an und ändern die Differenz kaum.

(c) Zwei der folgenden Gründe mit Begründung geben volle Punkte. *Genauigkeit:* Modelle nutzen Details in der Mitte sehr langer Kontexte schlechter (Lost in the Middle), ein kurzer, passender Kontext ist robuster. *Betrieb:* Die Dokumentation ändert sich und wächst, irgendwann passt sie nicht mehr ins Kontextfenster. *Zugriffsrechte:* Nur wenn Abschnitte gezielt ausgewählt werden, lässt sich steuern, wer welches Dokument sehen darf. *Wartezeit:* Rechen- und Wartezeit wachsen mit der Zahl der Tokens, denn jede Frage verarbeitet den ganzen Text neu. *Quellen:* Wenige Chunks mit Kennung (Q1, Q2) machen jede Aussage nachprüfbar.

> **Typischer Fehler**
>
> Den Faktor 10^6 vergessen und auf 300 000 US-Dollar pro Frage kommen. Prüfe jedes Ergebnis kurz auf Plausibilität: Eine Frage an ein Sprachmodell kostet Bruchteile eines Dollars.

Bewertung: (a) 1 P. (b) Kosten pro Frage und Monat 1 P, Faktor mit Begründung zu den Output-Tokens 1 P. (c) je Grund mit Begründung 1 P. Folgefehler aus (a) werden in (b) nicht abgezogen.

Lösung zu Aufgabe 2 Chunking mit Überlappung

(a) Wegen $s > 0$ gilt

$$js + g \geq L \iff j \geq \frac{L-g}{s}.$$

Für $j < (L-g)/s$ ist $js + g < L$, die Schleife läuft also weiter. Die kleinste ganze Zahl mit $j \geq (L-g)/s$ ist $j^* = \lceil (L-g)/s \rceil$, bei $L > g$ ist sie mindestens 1. Die Chunks tragen die Nummern $0, 1, \ldots, j^*$, das sind $j^* + 1$ Stück:

$$\boxed{n = \left\lceil \frac{L-g}{s} \right\rceil + 1}.$$

Die $+1$ kommt also daher, dass ab $j = 0$ gezählt wird. Für $o \geq g$ wäre $s \leq 0$: Das Fenster rückt nicht vor oder sogar zurück, die Schleife endet nie, und die Formel teilt durch null oder eine negative Zahl. Deshalb muss $o < g$ gelten.

(b) $s = 600 - 100 = 500$ und

$$n = \left\lceil \frac{5000 - 600}{500} \right\rceil + 1 = \lceil 8{,}8 \rceil + 1 = 9 + 1 = \boxed{10}.$$

Startpositionen: $0, 500, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500$. Probe mit der Stoppbedingung: Für $j = 8$ ist $4000 + 600 = 4600 < 5000$, es geht weiter. Für $j = 9$ ist $4500 + 600 = 5100 \geq 5000$, Schluss. Der letzte Chunk umfasst die Positionen 4500 bis 4999, er hat also nur $\boxed{500}$ Zeichen.

(c) Summe der Längen: $9 \cdot 600 + 500 = \boxed{5900}$ Zeichen, das ist das $5900/5000 = \boxed{1{,}18}$-Fache des Texts. Je zwei benachbarte Chunks überlappen sich um $o = 100$ Zeichen, auch das letzte Paar: Chunk 8 reicht bis 4599, Chunk 9 beginnt bei 4500. Bei 9 Paaren stehen $9 \cdot 100 = \boxed{900}$ Zeichen in genau zwei Chunks, also 18 % des Texts. Probe: $5000 + 900 = 5900$. Kein Zeichen steht in drei Chunks, weil $o < s$ ist.

(d) *Beweis.* Die Textstelle umfasse die Positionen a bis $a + \ell - 1$ mit $\ell \leq o$ und $a + \ell \leq L$. Setze $j = \min(\lfloor a/s \rfloor,\ n - 1)$. Dann gilt $js \leq a$, der Chunk beginnt also nicht zu spät.

Fall 1, Chunk j ist nicht der letzte. Dann ist $j = \lfloor a/s \rfloor$, also $a < (j + 1)s$, und der Chunk endet bei $js + g - 1$. Es folgt $a + \ell - 1 < (j + 1)s + o - 1 = js + g - 1$, die Textstelle endet also vor dem Ende des Chunks. *Fall 2, Chunk j ist der letzte.* Er reicht bis zum Textende $L - 1$, und die Textstelle endet bei $a + \ell - 1 \leq L - 1$. In beiden Fällen liegt die Textstelle vollständig in Chunk j. □

Satz ab Position 490: Er umfasst die Positionen 490 bis 609. Chunk 0 endet bei 599, es fehlen 600 bis 609. Chunk 1 beginnt erst bei 500, es fehlen 490 bis 499. Alle weiteren Chunks beginnen bei 1000 oder später. Der Satz ist also **zerschnitten**. Das widerspricht dem Beweis nicht, denn $120 > o = 100$.

Nach dem Beweis garantiert eine Überlappung von $\boxed{o \geq 120}$, dass kein Satz mit bis zu 120 Zeichen zerschnitten wird, etwa $g = 600$ und $o = 120$, also $s = 480$. Feinheit für Genaue: Weil die Positionen ganzzahlig sind, reicht sogar $o = 119$. Die Antwort $o \geq 120$ gibt volle Punkte.

Typischer Fehler

Mit der Chunk-Größe statt der Schrittweite weiterrücken, also Starts bei $0, 600, 1200, \ldots$ Dann gibt es keine Überlappung, und Sätze an den Grenzen gehen verloren. Ebenso häufig: die $+1$ vergessen. Dann endet der letzte Chunk bei Position 4599, und die letzten 400 Zeichen des Texts fehlen.

Bewertung: (a) Umformung der Stoppbedingung 1 P, Erklärung der $+1$ und von $o < g$ 1 P. (b) n 1 P, Startpositionen und Länge des letzten Chunks 1 P. (c) Summe und Faktor 1 P, Zahl der doppelten Zeichen 1 P. (d) Beweis mit beiden Fällen 1 P, Satz ab 490 und $o \geq 120$ 1 P.

Lösung zu Aufgabe 3 Mit Kosinus suchen, dann genauer sortieren

(a) $\|e_3\|^2 = 0{,}48^2 + 0{,}6^2 + 0{,}64^2 = 0{,}2304 + 0{,}36 + 0{,}4096 = 1$, also $\|e_3\| = 1$. Weil alle Vektoren normiert sind, ist $s_i = e_i \cdot q$ die Kosinus-Ähnlichkeit:

$$s_1 = 0{,}8 \cdot 0{,}6 + 0{,}6 \cdot 0 + 0 \cdot 0{,}8 = 0{,}48, \qquad s_2 = 0 + 0 + 0{,}8 \cdot 0{,}8 = 0{,}64,$$
$$s_3 = 0{,}48 \cdot 0{,}6 + 0 + 0{,}64 \cdot 0{,}8 = 0{,}288 + 0{,}512 = 0{,}80, \qquad s_4 = 0{,}6 \cdot 0{,}6 + 0 + 0 = 0{,}36.$$

Also ist $s = (0{,}48;\ 0{,}64;\ 0{,}80;\ 0{,}36)$, die Rangfolge lautet **3, 2, 1, 4**.

(b) $q \cdot e_5 = 0{,}6 \cdot 1{,}2 + 0 \cdot 1{,}6 + 0{,}8 \cdot 0 = \boxed{0{,}72}$ und $\|e_5\| = \sqrt{1{,}44 + 2{,}56} = 2$, also $\cos(q, e_5) = 0{,}72/(1 \cdot 2) = \boxed{0{,}36}$.

Mit dem Skalarprodukt läge Chunk 5 wegen $0{,}80 > 0{,}72 > 0{,}64$ auf **Platz 2**. Richtig ist der Kosinus 0,36: Es gilt $e_5 = 2\,e_4$, beide zeigen in dieselbe Richtung. Chunk 5 gehört also gleichauf mit Chunk 4 auf den **letzten Platz**. Das Skalarprodukt ist nur bei normierten Vektoren gleich dem Kosinus, sonst gewinnen lange Vektoren. Deshalb normiert das Notebook Chunks und Frage mit `normalize_embeddings=True`.

(c) (i) $120\,000 \cdot 15\,\text{ms} = 1\,800\,000\,\text{ms} = 1800\,\text{s} = \boxed{30\,\text{min}}$ pro Frage.

(ii) $25 \cdot 15\,\text{ms} = 375\,\text{ms}$ für das Reranking, mit der ersten Stufe $\boxed{415\,\text{ms}}$.

(iii) $40 + k \cdot 15 \le 1000 \iff k \le 960/15 = 64$. Höchstens $\boxed{64}$ Kandidaten.

Der Grund für den Unterschied: Beim Bi-Encoder sind alle Chunk-Vektoren vorab gespeichert, pro Frage bleiben ein Durchlauf für q und billige Skalarprodukte. Der Cross-Encoder braucht für jedes Paar einen vollen Durchlauf.

(d) Das Reranking sortiert nur die 25 Kandidaten der ersten Stufe neu, neue Chunks kommen nicht hinzu. Fehlt bei einer Frage der relevante Chunk unter den 25, kann er nach dem Reranking auch nicht unter den ersten 5 sein. Die Fragen mit Treffer in den Top 5 nach dem Reranking sind also eine Teilmenge der Fragen mit Treffer in den Top 25 der ersten Stufe. Daraus folgt $\text{Recall@5}_{\text{nachher}} \le \text{Recall@25}_{\text{vorher}}$. Die erste Stufe muss deshalb genug Kandidaten liefern.

> **Typischer Fehler**
>
> Die Rangfolge aufsteigend angeben (Chunk 4, 1, 2, 3). Genau das liefert `np.argsort` ohne Umdrehen, der häufigste Fehler in TODO 4.

Bewertung: (a) Länge von e_3 und s 1 P, Rangfolge 1 P. (b) Skalarprodukt und Kosinus 1 P, beide Plätze mit Begründung 1 P. (c) je Teil (i), (ii), (iii) 1 P. (d) 1 P für das Argument „keine neuen Kandidaten“.

Lösung zu Aufgabe 4 BM25 von Hand

(a) Die beiden idf-Varianten:

Wort	n_t	$\ln(N/n_t)$	$\ln \frac{N-n_t+0{,}5}{n_t+0{,}5}$
e21	1	$\ln 10 = 2{,}303$	$\ln(9{,}5/1{,}5) = \ln 6{,}333 = 1{,}846$
kühlflüssigkeit	2	$\ln 5 = 1{,}609$	$\ln(8{,}5/2{,}5) = \ln 3{,}4 = 1{,}224$
prüfen	4	$\ln 2{,}5 = 0{,}916$	$\ln(6{,}5/4{,}5) = \ln 1{,}444 = 0{,}368$

In beiden Varianten zählt das seltene Wort *e21* am meisten. Die BM25-Variante drückt häufige Wörter stärker: *prüfen* bekommt nur noch 0,368.

(b) TF-IDF-Scores:

$$\begin{aligned} \text{C1:}&\quad 1 \cdot 2{,}303 + 1 \cdot 1{,}609 + 5 \cdot 0{,}916 = 8{,}49,\\ \text{C2:}&\quad 2 \cdot 1{,}609 + 3 \cdot 0{,}916 = 5{,}97,\\ \text{C3:}&\quad 8 \cdot 0{,}916 = 7{,}33. \end{aligned}$$

Rangfolge mit TF-IDF: **C1, C3, C2**. Die Checkliste C3 überholt C2 nur durch achtmal *prüfen*.

(c) Längenfaktor $\lambda = 1 - b + b\,|D|/\text{avgdl}$: für C1 $0{,}25 + 0{,}75 \cdot 1{,}5 = 1{,}375$, für C2 genau 1, für C3 $0{,}25 + 0{,}75 \cdot 2 = 1{,}75$. Damit ist $k_1\lambda = 2{,}0625$, 1,5 und 2,625, und mit $k_1 + 1 = 2{,}5$:

$$\begin{aligned} &\text{C1:} \quad f(1) = \tfrac{2{,}5}{1+2{,}0625} = 0{,}816, \qquad f(5) = \tfrac{12{,}5}{5+2{,}0625} = 1{,}770, \\ &\text{C2:} \quad f(2) = \tfrac{5}{2+1{,}5} = 1{,}429, \qquad f(3) = \tfrac{7{,}5}{3+1{,}5} = 1{,}667, \\ &\text{C3:} \quad f(8) = \tfrac{20}{8+2{,}625} = 1{,}882. \end{aligned}$$

Scores:

$$\begin{aligned} &\text{BM25(C1)} = 1{,}846 \cdot 0{,}816 + 1{,}224 \cdot 0{,}816 + 0{,}368 \cdot 1{,}770 = 1{,}507 + 0{,}999 + 0{,}651 = \boxed{3{,}157}, \\ &\text{BM25(C2)} = 1{,}224 \cdot 1{,}429 + 0{,}368 \cdot 1{,}667 = 1{,}748 + 0{,}613 = \boxed{2{,}361}, \\ &\text{BM25(C3)} = 0{,}368 \cdot 1{,}882 = \boxed{0{,}692}. \end{aligned}$$

Rangfolge mit BM25: **C1, C2, C3**.

(d) Mit $\ln(N/n_t)$ und den Faktoren f aus (c):

$$\text{C2: } 1{,}609 \cdot 1{,}429 + 0{,}916 \cdot 1{,}667 = 2{,}299 + 1{,}527 = 3{,}826, \qquad \text{C3: } 0{,}916 \cdot 1{,}882 = 1{,}725.$$

C2 liegt schon mit den einfachen Gewichten vor C3. Der Platztausch kommt also vom **Faktor** f: Die Sättigung macht aus acht Vorkommen nur 1,882 statt 8, die Längennormierung dämpft zusätzlich. Die Sättigung ist der größere Effekt. Ohne Längennormierung ($b = 0$) wäre $f(8) = 20/9{,}5 = 2{,}105$, immer noch weit unter 8.

(e) (i) Für $|D| = \text{avgdl}$ ist die Klammer $1 - b + b = 1$, also $f(1) = \frac{1\cdot(k_1+1)}{1+k_1} = 1$.

(ii) Mit λ wie in (c) ist $f(\text{tf}) = \dfrac{k_1 + 1}{1 + k_1\lambda/\text{tf}}$. Für $\text{tf} \to \infty$ geht $k_1\lambda/\text{tf} \to 0$, also $f \to k_1 + 1 = 2{,}5$.

(iii) Gesucht ist $r = |D|/\text{avgdl}$ mit

$$\frac{k_1 + 1}{1 + k_1\lambda} = 0{,}5 \iff 1 + k_1\lambda = 2(k_1 + 1) = 5 \iff \lambda = \frac{4}{1{,}5} = \frac{8}{3}.$$

Aus $0{,}25 + 0{,}75\,r = 8/3$ folgt $0{,}75\,r = 29/12$ und $\boxed{r = 29/9 \approx 3{,}22}$. Ein Chunk mit gut der dreifachen mittleren Länge zählt ein einzelnes Vorkommen nur noch halb.

> **Typischer Fehler**
>
> Den Längenfaktor vergessen und für alle Chunks mit $\lambda = 1$ rechnen. Dann bekommt C1 für *e21* den Faktor 1 statt 0,816, und der lange Chunk C3 wird zu stark bewertet. Ebenfalls häufig: $\log_{10}$ statt ln verwenden.

Bewertung: (a) je idf-Variante für alle drei Wörter 1 P. (b) 1 P. (c) Faktoren f 1 P, Scores 1 P, Rangfolge 1 P. (d) 1 P. (e) (i) und (ii) zusammen 1 P, (iii) 1 P. Rundungsabweichungen in der dritten Nachkommastelle kosten keine Punkte, Folgefehler aus (a) werden nicht doppelt abgezogen.

Lösung zu Aufgabe 5 Hybride Suche messen und Fehler finden

(a) Jede Liste vergibt $1/(60 + \text{rang})$ Punkte:

Chunk	Rang Vektor	Rang BM25	RRF-Punkte
S	1	3	$\frac{1}{61} + \frac{1}{63} = 0{,}016393 + 0{,}015873 = 0{,}03227$
F	5	2	$\frac{1}{65} + \frac{1}{62} = 0{,}015385 + 0{,}016129 = 0{,}03151$
W	3	5	$\frac{1}{63} + \frac{1}{65} = 0{,}015873 + 0{,}015385 = 0{,}03126$
P	fehlt	1	$\frac{1}{61} = 0{,}01639$
K	2	fehlt	$\frac{1}{62} = 0{,}01613$
L	4	fehlt	$\frac{1}{64} = 0{,}01563$
Y	fehlt	4	$\frac{1}{64} = 0{,}01563$

Fusionierte Rangfolge: **S, F, W, P, K**, dann L und Y gleichauf. F steht auf Rang 2 und ist damit **im Prompt**. Mit der Vektorsuche allein stünde F auf Rang 5 und fiele bei $k = 4$ heraus. Chunks, die in beiden Listen auftauchen, sammeln zweimal Punkte. Der BM25-Spitzenreiter P, in dem von den Fragewörtern nur „bedeutet“ vorkommt, dafür dreimal, rutscht auf Rang 4.

(b) Treffer in den Top 1: Vektorsuche F1, F6; hybrid F1, F6. Treffer in den Top 4: Vektorsuche F1, F3, F5, F6; hybrid zusätzlich F2.

$$\begin{aligned}
\text{Recall@1:} \quad & \text{Vektor } \tfrac{2}{6} = 33{,}3\,\%, \qquad \text{Hybrid } \tfrac{2}{6} = 33{,}3\,\%, \\
\text{Recall@4:} \quad & \text{Vektor } \tfrac{4}{6} = 66{,}7\,\%, \qquad \text{Hybrid } \tfrac{5}{6} = 83{,}3\,\%, \\
\text{MRR}_{\text{Vektor}} &= \tfrac{1}{6}\left(1 + \tfrac{1}{5} + \tfrac{1}{2} + 0 + \tfrac{1}{3} + 1\right) = \tfrac{3{,}033}{6} = \boxed{0{,}506}, \\
\text{MRR}_{\text{Hybrid}} &= \tfrac{1}{6}\left(1 + \tfrac{1}{2} + \tfrac{1}{3} + \tfrac{1}{7} + \tfrac{1}{2} + 1\right) = \tfrac{3{,}476}{6} = \boxed{0{,}579}.
\end{aligned}$$

(c) Eine Frage macht $1/6 \approx 16{,}7$ Prozentpunkte aus. Der Unterschied bei Recall@4 ist genau eine Frage (F2). Das ist **nicht belastbar**: Schon bei 15 Testfragen gelten ein oder zwei Fragen Unterschied nicht als Befund, bei sechs Fragen erst recht nicht.

Inhaltlich spricht trotzdem etwas für die hybride Suche: Die deutlichsten Gewinne gibt es bei F2 (E45, von Rang 5 auf 2) und F4 (HTD-3310, von „fehlt“ auf Rang 7), also bei den beiden Fragen mit Kennung. Das passt zum Mechanismus aus der Lerneinheit, denn BM25 findet Kennungen, Embeddings kaum. Bei F3 und F5 verschiebt sich der Treffer nur um einen Rang und bleibt im Prompt. Nächster Schritt: die verfehlten Fragen einzeln ansehen (F4 fehlt auch hybrid im Prompt), das Testset um weitere Fragen mit und ohne Kennungen erweitern und erneut messen. Eine gute Formulierung fürs Protokoll: „Hybrid verbessert den Rang bei beiden Kennungsfragen, insgesamt noch nicht belastbar.“

(d) *Fall 1: Auswahl.* Die Antwort „Retrieval“ mit Verweis auf den Rang gibt ebenfalls volle Punkte, denn die Lerneinheit nennt ihr Beispiel mit Rang 7 einen Retrieval-Fehler. Ein Chunk mit der Antwort existiert: Die Ersatzteiltabelle von M-03 enthält HTD-3310 (Rücklauffilter Hydraulik, Regal C-04). Er liegt aber auf Rang 7 und ist bei $k = 4$ nicht im Prompt. Das Modell hat sich korrekt verhalten und nichts erfunden. Abhilfe: k auf mindestens 7 erhöhen (kostet etwa $3 \cdot 200$ Tokens mehr), einen Cross-Encoder über die Top 20 sortieren lassen, die Frage im Testset behalten. Ein stärkeres Modell hilft nicht.

Fall 2: Generierung. Die richtige Zeile (500 Betriebsstunden) steht vollständig in Q1. Das Modell hat die benachbarte Zeile zum Spannfutter gelesen. Die Quellenangabe [Q1] macht den Fehler sofort nachprüfbar. Abhilfe: den System-Prompt schärfen (etwa die Tabellenzeile

wörtlich zitieren lassen), ein anderes Modell testen und die Frage ins Testset aufnehmen, damit das Eval in Modul 9 sie automatisch prüft.

Typischer Fehler

Beim MRR die Frage ohne Treffer weglassen. Dann teilst du für die Vektorsuche durch 5 und erhältst fälschlich $3{,}033/5 \approx 0{,}607$ statt 0,506. Eine Frage ohne Treffer zählt mit 0 im Mittelwert.

Bewertung: (a) RRF-Punkte 2 P (je Fehler 0,5 P Abzug), Rangfolge mit Antwort zu $k = 4$ 1 P. (b) Recall@1 und Recall@4 1 P, je MRR 1 P. (c) Prozentpunkte und Urteil 1 P, inhaltliches Argument mit nächstem Schritt 1 P. (d) je Fall mit Abhilfe 1 P.

Lösung zu Aufgabe 6 Wozu die 60 in Reciprocal Rank Fusion? (Zusatz)

(a) Q liegt vor P, wenn

$$\frac{2}{c+3} > \frac{1}{c+1} + \frac{1}{c+8} = \frac{2c+9}{(c+1)(c+8)}.$$

Alle Nenner sind positiv, Multiplizieren erhält die Richtung:

$$\begin{aligned} 2(c+1)(c+8) > (2c+9)(c+3) &\iff 2c^2 + 18c + 16 > 2c^2 + 15c + 27 \\ &\iff 3c > 11 \iff \boxed{c > 11/3 \approx 3{,}67}. \end{aligned}$$

Probe: Für $c = 0$ ist P $= 1 + \frac{1}{8} = 1{,}125$ und Q $= \frac{2}{3} \approx 0{,}667$, P liegt vorn. Für $c = 60$ ist P $= \frac{1}{61} + \frac{1}{68} = 0{,}01639 + 0{,}01471 = 0{,}03110$ und Q $= \frac{2}{63} = 0{,}03175$, Q liegt vorn.

(b) Es gilt $\frac{1}{c+r} = \frac{1}{c} \cdot \frac{1}{1+r/c} \approx \frac{1}{c}\left(1 - \frac{r}{c}\right)$. Für einen Chunk in m Listen mit den Rängen $r_1, \ldots, r_m$ folgt

$$\mathrm{RRF}_c \approx \frac{m}{c} - \frac{r_1 + \cdots + r_m}{c^2}.$$

Der erste Term ist um den Faktor c größer als der zweite, solange die Rangsummen klein gegen c sind. Also entscheidet zuerst die Zahl m der Listen. Bei gleichem m gewinnt die kleinere Rangsumme. Probe an 5(a): S (Rangsumme 4), F (7) und W (8) stehen in beiden Listen und kommen in dieser Reihenfolge zuerst. Danach folgen die Chunks aus nur einer Liste nach ihrem Rang: P (1), K (2), dann L und Y (je 4). Ohne Konstante ($c = 0$) entschiede dagegen ein einzelner Spitzenplatz, wie bei P in (a). Bei gleicher Rangsumme entscheidet der nächste Term der Näherung.